# Badr Al Jerf — Tasks (Backend platform, Auth, Users, Profile, Swagger)

## Purpose

You build the backend foundation that makes the app secure: the Nest app setup, Swagger at `/docs`, register/login with JWT, the **global** authentication guard, users, and profile statistics.

Read first: `docs/SPECIFICATION.md`, `docs/API-CONTRACT.md` (sections 1, 2, 4, 8), `docs/TEAM-OWNERSHIP.md`.

## 1. Files you own (only you edit these)

| File | What goes in it |
|---|---|
| `backend/src/main.ts` | Create app, global prefix `api/v1`, global `ValidationPipe` (`whitelist`, `forbidNonWhitelisted`, `transform`), CORS from `CORS_ORIGIN`, Swagger setup at `/docs` with Bearer auth and `persistAuthorization: true` |
| `backend/src/app.module.ts` | `ConfigModule` (global), `MongooseModule.forRootAsync` with `MONGODB_URI`, `AuthModule`, `UsersModule`, global guard as `APP_GUARD`. Others add their module line (see coordination §3) |
| `backend/src/config/env.validation.ts` | Validate required env vars at startup (`MONGODB_URI`, `JWT_SECRET`, |

| File | What goes in it |
| --- | --- |
| | `JWT_EXPIRES_IN`, `PORT`, `CORS_ORIGIN`) |
| `backend/src/auth/auth.module.ts` | Register `JwtModule` (async, from config), controller, service, guard |
| `backend/src/auth/auth.controller.ts` | `POST /auth/register`, `POST /auth/login` (public), `GET /auth/me` |
| `backend/src/auth/auth.service.ts` | Hash with bcrypt, verify password, sign token, build login response |
| `backend/src/auth/jwt-auth.guard.ts` | Global guard: skip routes marked public, verify Bearer token, set `request.user = { userId, username }`, else 401 `UNAUTHORIZED` |
| `backend/src/auth/public.decorator.ts` | `@Public()` metadata used **only inside your auth controller** |
| `backend/src/auth/dto/register.dto.ts` | Register validation + `@ApiProperty` |
| `backend/src/auth/dto/login.dto.ts` | Login validation |
| `backend/src/auth/dto/auth-response.dto.ts` | Swagger shape of the login/register response |
| `backend/src/users/users.module.ts` | Users schema/model, controller, service |
| `backend/src/users/schemas/user.schema.ts` | User fields from the contract, unique indexes, never select `passwordHash` by default |
| `backend/src/users/users.controller.ts` | `PATCH /users/me`, `GET /users/:id/profile` |
| `backend/src/users/users.service.ts` | Create/find users (used by your auth service), update profile, numeric id |

| File | What goes in it |
| --- | --- |
| | helper ( `counters` key `users` ) |
| `backend/src/users/profile-stats.service.ts` | Read `questions` , `interviews` , `attempts` collections (native driver, contract §8 fields) to compute stats |
| `backend/src/users/dto/update-me.dto.ts` | `displayName` , `bio` validation |
| `backend/src/users/dto/user-response.dto.ts` | `PrivateUser` , `PublicUser` , `Profile` Swagger shapes |
| `backend/test/auth.e2e-spec.ts` | Your tests (see §5) |
| `backend/test/users.e2e-spec.ts` | Your tests |
| `backend/.env.example` | Variable names with safe example values only |
| `.gitignore` | Ignore `.env` , `node_modules` , `dist` , coverage |
| `README.md` → sections *Backend setup*, *Environment variables*, *Swagger (/docs)* | Only these headings |

## 2. Files you must NOT edit

- Anything under `backend/src/questions/` , `interviews/` (Shiam), `attempts/` (Esmaeel), and all of `frontend/` .
- Lines other people add in `app.module.ts` — keep them when you edit the file later.
- `docs/API-CONTRACT.md` alone.

## 3. Implementation order (no one to wait for)

1. `main.ts` , `app.module.ts` , env validation, `.env.example` , `.gitignore` . Check `/docs` opens with an empty Swagger page.

2. User schema + users service (create, find by username/email, find by id, numeric id helper).
3. Register/login DTOs → auth service → auth controller. Check in Swagger that login returns a token.
4. Global guard + `@Public()`. Check: `/auth/me` gives 401 without token and works after "Authorize" in Swagger.
5. `PATCH /users/me` and `GET /users/:id/profile`.
6. Profile stats: since other collections may be empty or not exist yet, your code must return zeros / `null` safely. In tests, insert raw documents into `questions`, `interviews`, `attempts` with the contract §8 shape.
7. Swagger polish: every endpoint has tags, response types, and 400/401/403/404/409 responses where relevant.

## 4. Things to think about (discuss with the team, then decide)

- How will the guard know a route is public without other modules importing anything? (Hint: only *your* routes are public.)
- Login must not reveal whether the username exists. Why does that matter?
- `averagePercentage` without aggregation: what is the minimum data you need to read from `attempts`?
- What should happen to the token if a user is deleted? (Not required, but be ready to explain.)

## 5. Completion checklist

- [ ] `/docs` shows every auth/users endpoint; "Authorize" works and survives a page reload.
- [ ] Every non-public route in the **whole app** returns 401 `UNAUTHORIZED` without/with a wrong/expired token.
- [ ] Duplicate username → 409 `USERNAME_TAKEN`; duplicate email → 409 `EMAIL_TAKEN`; usernames/emails stored lowercase.
- [ ] Wrong password and unknown user give the same 401 `INVALID_CREDENTIALS`.

- [ ] `passwordHash` never appears in any response or log.
- [ ] Unknown body fields are rejected (400).
- [ ] Profile stats correct for: a user with nothing, a user with only in-progress attempts, a user with several ended attempts.
- [ ] `PublicUser` has no email.
- [ ] Secrets only in `.env`; app refuses to start if `JWT_SECRET` is missing.

## 6. Coordination

- You don't wait for anyone. When Shiam/Esmaeel open PRs, they add their module line to `app.module.ts`; review that it's only that line.
- Tell everyone once the global guard is merged so they can try their endpoints in `/docs` with a real token.